# Advance Educational Activities Pvt. Ltd.

# Unit 5: Trees

## 5.1 — Introduction to Trees & Terminology

Where data stops moving in a line… and starts branching into possibilities.

### 1️⃣ What Is a Tree?

A **Tree** is a *non-linear*, *hierarchical* data structure made up of nodes connected through edges.

Unlike arrays or linked lists where data moves in a **single line**,
trees expand in **multiple directions**, enabling rich, branching relationships.

```
        A
      /   \
    B       C
   / \     / \
  D   E   F   G
```

**Essence:** Trees model relationships that are *not sequential*, but *hierarchical*.

### 2️⃣ Why Not Arrays or Linked Lists?

Linear structures fail when:

- Data has **parent–child** relationships
- You need hierarchical navigation
- You want to represent structured, branching information
- You need to perform fast searching (O(log n)) in balanced trees
- Recursion-heavy traversal is required

Examples linear structures CANNOT represent well:

❌ File-system directories

❌ Organization structures (CEO → Managers → Teams)

❌ Expression hierarchies

❌ HTML DOM trees

❌ Decision trees

Trees shine where linear structures collapse.

**© Advanced Educational Activities Pvt. Ltd. 95B, Siddamsetty Complex, Park Lane, Secunderabad – 500003.** All rights reserved. No part of this material may be reproduced, in any form or by any means, without permission in writing. This course material is only for the use of bonafide students of **Advanced Educational Activities Pvt. Ltd. (T.I.M.E.)** and its licensees/franchisees and is not for sale. (44 pages) (cael/caem) SM2132505/1

## 3️⃣ Trees Are Non-Linear (Why This Matters)

**Linear structures:**

- Single path
- One successor per node
- Simpler, but limited

**Non-linear structures:**

- Multiple paths
- A node can have **many children**
- Much richer representation
- Ideal for search, hierarchy, branching, recursion

Trees allow exploration in multiple directions at once.

## 4️⃣ Basic Terminology of Trees

Let's define every fundamental word in tree theory — clearly, visually, perfectly.

**Node**

The fundamental unit that holds data.

```
   [A]
```

**Root**

Topmost node of a tree. The origin.

```
 Root → A
        |
        B
```

A tree MUST have one root.

**Parent**

A node that has child nodes.

A is parent of B, C

**Child**

A node that descends from a parent node.

B and C are children of A

**Sibling**

Nodes with the same parent are siblings.

```
B and C
↑      ↑
Children of A → siblings
```

**Leaf Node (Terminal Node)**

A node with no children.

```
   B
  / \
 D   E   ← Leaf nodes
```

**Internal Node**

Any node that is NOT a leaf.

**Degree of a Node**

Number of children a node has.

**Example:**

A → degree 2

B → degree 2

D → degree 0

**Depth of Node**

Distance from the *root* to the node.

Root depth = 0
Child depth = parent depth + 1

**Level of a Node**

Depth + 1
(or simply: nodes at the same horizontal row)

Level 1 → Root

Level 2 → Children of root

Level 3 → Grandchildren

**Height of a Node**

Longest downward path from the node to a leaf.

Height of leaf = 0
Height of tree = height of root

**Subtree**

A node + all its descendants.

Subtree rooted at B:

```
   B
  / \
 D   E
```

**Ordered vs Unordered Trees**

**Ordered Tree**

- Children follow a positional order (left, right).
- Binary trees = ordered trees.

**Unordered Tree**

Children have no positional meaning.

## 5️⃣ General Properties of Trees

- Trees are **connected** graphs with no cycles
- If a tree has *n* nodes, it has **n – 1 edges**
- There is **exactly one path** between any two nodes
- Each child node has exactly **one parent**
- Trees grow downward from the root
- Recursive by nature (tree inside tree inside tree…)

## 6️⃣ Real-World Motivation (Why Trees Exist Everywhere)

Trees naturally model hierarchical or recursive systems:

**1. File Systems**

```
Root
├── Documents
│     ├── Notes
│     └── Reports
└── Pictures
```

**2. HTML / XML DOM Structure**

```
<html>
  <body>
    <div>
      <p>
```

**3. Organizational Structures**

CEO → Manager → Team Leads → Developers

4. AI Decision Trees / Game Trees

5. Indexing & Searching (BST, Trie, B-Tree)

6. Trees are the backbone of nearly every system with hierarchy.

## Quick Summary

- Trees are non-linear, hierarchical structures
- Used when linear structures are not enough
- Nodes contain **parent-child** relationships

- Core terms: root, parent, child, leaf, sibling, depth, height
- Trees power file systems, DOM, decision trees, compilers, databases

**Mantra:**

"When data stops walking in a line… it begins to grow like a tree." 🌳

## 5.2 — Binary Tree Fundamentals

The foundation of all hierarchical data thinking.

### 1️⃣ Definition: What Is a Binary Tree?

A **Binary Tree (BT)** is a hierarchical, non-linear data structure where:

- Every node can have **0, 1, or 2 children**
- Children are referred to as:
  - Left child
  - Right child
- Structure is **recursive** — each subtree is itself a binary tree.

It is *not* necessarily ordered, balanced, or complete —
it is simply the most general form of a two-child tree.

### 2️⃣ Node Structure (Conceptual)

A Binary Tree node typically contains:

```
data
left child pointer
right child pointer
```

In Java (Linked Representation):

```
class Node {
    int data;
    Node left;
    Node right;

    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
```

### 3️⃣ Types of Binary Trees

Understanding these variations helps in efficiency, traversals, and applications.

### 3.1 Full Binary Tree

A Binary Tree where:

- Every node has either 0 or 2 children
- No node has exactly one child

Example:

```
      A
    /   \
   B     C
        / \
       D   E
```

All parents have exactly 2 children.

### 3.2 Perfect Binary Tree

A stronger form of full tree.

Conditions:

- All internal nodes have 2 children
- All leaf nodes are at the **same level**
- It is a completely filled structure

Example:

```
          A
        /   \
      B       C
     / \     / \
    D   E   F   G
```

Properties:

- Nodes = 2^(h+1) − 1
- Leaves = 2^h

### 3.3 Complete Binary Tree (Definition Only)

A Binary Tree in which:

- All levels are completely filled **except possibly the last level**
- The last level has nodes from left to right without gaps

Example:

```
          A
        /   \
      B       C
     / \     /
    D   E  F
```

This structure is the foundation of **heaps**.

### 3.4 Skewed Binary Tree

A tree with nodes only on one side.

Left-Skewed:

```
      A
     /
    B
   /
  C
 /
D
```

Right-Skewed:

```
A
 \
  B
   \
    C
     \
      D
```

Equivalent to a linked list → worst-case behavior.

## 4️⃣ Array Representation Theory

Binary Trees can be stored in arrays using **index relationships**:

**For a node at index `i`:**

- Left child = `2*i + 1`
- Right child = `2*i + 2`
- Parent = `(i - 1) / 2`

**Example array:**

```
Index:  0   1   2   3   4   5   6
Value:  A   B   C   D   E   F   G
```

**Represents:**

```
          A (0)
       /       \
   B (1)       C (2)
   / \         / \
D(3) E(4)   F(5) G(6)
```

Best for Complete Binary Trees (CBT, heaps).

Not good for:

- Skewed trees
- Sparse trees

## 5️⃣ Linked List Representation

Using Node objects with pointers:

```
class Node {
    int data;
    Node left;
    Node right;
}
```

**Structure:**

```
       10
      /  \
    20    30
   / \      \
 40  50     60
```

Each node dynamically allocates child nodes.

This representation is:

- Flexible
- Dynamic
- Space-efficient for sparse trees

## 6️⃣ Basic Examples & Diagrams

**Example Binary Tree:**

```
        15
       /  \
     10    20
    / \     \
   8  12    25
```

**Another Example:**

```
       A
     /   \
    B     C
         /
        D
```

**Example of a non-complete general binary tree:**

```
      X
     / \
    Y   Z
   /     \
  P       Q
```

## Quick Summary

- Binary Tree: max **2 children per node**
- Node = data + left + right
- Types:
  - Full
  - Perfect
  - Complete
  - Skewed
- Array representation → uses index math
- Linked representation → dynamic pointers
- Used everywhere: heaps, parsing, compilers, expression evaluation

**Mantra:**

"Binary Trees are the skeleton of hierarchical logic — simple, yet endlessly powerful."

# 5.3 — Binary Tree Traversals

How we walk through a tree.

## 1️⃣ Why Traversal?

A binary tree is a hierarchical structure — you **cannot** access it the way you access arrays (`arr[i]`) or linked lists (`node.next`).

Traversal gives us a **systematic method** to visit all nodes **exactly once** in a well-defined order.

We need traversal for:

- Searching a node
- Printing or processing tree data
- Constructing / reconstructing trees
- Evaluating expression trees
- Serializing trees (save → reload)

Traversal = the language of trees.

## 2️⃣ Two Major Traversal Families

### A. Depth-First Search (DFS)

Visits as deep as possible before backtracking.

**3 styles:**

- **Preorder** → Root → Left → Right
- **Inorder** → Left → Root → Right
- **Postorder** → Left → Right → Root

### B. Breadth-First Search (BFS)

Visits level by level.
Also called **Level Order Traversal** (LOT).

## 3️⃣ Before Traversing: Node Structure (Linked Representation)

```
class Node {
    int data;
    Node left, right;

    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
```

## 4️⃣ Preorder Traversal

### Order

Root → Left → Right

### Recursive Code

```
void preorder(Node root) {
    if (root == null) return;
    System.out.print(root.data + " ");
    preorder(root.left);
    preorder(root.right);
}
```

### Iterative Code (Using Stack)

```
void preorderIterative(Node root) {
    if (root == null) return;

    Stack<Node> st = new Stack<>();
    st.push(root);
```

```
while (!st.isEmpty()) {
        Node curr = st.pop();
        System.out.print(curr.data + " ");

        if (curr.right != null) st.push(curr.right);
        if (curr.left != null) st.push(curr.left);
    }
}
```

When to Use Preorder?

- Clone/Copy a tree
- Convert tree → prefix notation
- Serialize a tree (store structure)

## 5️⃣ Inorder Traversal

**Order**

Left → Root → Right

**Recursive Code**

```
void inorder(Node root) {
    if (root == null) return;
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
}
```

**Iterative Code (Using Stack)**

```
void inorderIterative(Node root) {
    Stack<Node> st = new Stack<>();
    Node curr = root;

    while (curr != null || !st.isEmpty()) {
        while (curr != null) {
            st.push(curr);
            curr = curr.left;
        }
        curr = st.pop();
        System.out.print(curr.data + " ");
        curr = curr.right;
    }
}
```

When to Use Inorder?

- Very important: Prints BST in sorted order
- Used in tree reconstruction

- Common for expression tree evaluation

## 6️⃣ Postorder Traversal

**Order**

Left → Right → Root

**Recursive Code**

```
void postorder(Node root) {
    if (root == null) return;
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data + " ");
}
```

**Iterative Code (Two Stacks)**

```
void postorderIterative(Node root) {
    if (root == null) return;

    Stack<Node> s1 = new Stack<>();
    Stack<Node> s2 = new Stack<>();

    s1.push(root);

    while (!s1.isEmpty()) {
        Node curr = s1.pop();
        s2.push(curr);

        if (curr.left != null) s1.push(curr.left);
        if (curr.right != null) s1.push(curr.right);
    }

    while (!s2.isEmpty()) {
        System.out.print(s2.pop().data + " ");
    }
}
```

When to Use Postorder?

- Delete/free a tree
- Evaluate postfix expression trees
- Bottom-up construction (heaps, DP trees)

## 7️⃣ Level Order Traversal (BFS)

**Order:** Visit level by level from root:

Level 0 → Level 1 → Level 2 → …

**Code Using Queue**

```
void levelOrder(Node root) {
    if (root == null) return;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node curr = q.poll();
        System.out.print(curr.data + " ");

        if (curr.left != null) q.add(curr.left);
        if (curr.right != null) q.add(curr.right);
    }
}
```

When to Use Level Order?

- Shortest path in tree
- Building a complete binary tree
- BFS algorithms
- Serialization
- Heaps

## 8️⃣ Dry Run Example (Common Tree)

**If the tree is:**

```
        1
      /   \
     2     3
    / \   / \
   4   5 6   7
```

**Preorder:**

1 2 4 5 3 6 7

**Inorder:**

4 2 5 1 6 3 7

**Postorder:**

4 5 2 6 7 3 1

**Level Order:**

1 2 3 4 5 6 7

## 9️⃣ Traversal Pattern Memory Map

**Mnemonic love:**

PRE → Root first

IN → Root in middle

POST → Root last

**Or visual memory:**

PRE = R-L-R

IN = L-R-R

POST = L-R-R°

BFS = horizontal sweep
DFS = vertical depth exploration

## Quick Summary

- Traversal is essential to *visit and process* tree nodes
- DFS: Pre/In/Post with both recursive & iterative code
- BFS: Level Order using queue
- Inorder gives **sorted order** for BST
- Each traversal solves a different class of problems
- Pre = root first | In = root middle | Post = root last
- BFS is used for CBT, heaps, shortest paths

# 5.4 — Binary Tree Operations

Binary Tree operations are used everywhere: file systems, expression trees, game trees, compilers, AI search, and more.
Mastering these makes the foundation extremely strong.

## 1️⃣ Building a Binary Tree from User Input (Recursive)

**We construct a tree based on input like:**

Enter data: 1

Enter left of 1: 2

Enter left of 2: -1 (no node)

Enter right of 2: -1

Enter right of 1: 3

Enter left of 3: -1

Enter right of 3: -1

**Java Code**

```
class Node {
    int data;
    Node left, right;
    Node(int data) {
        this.data = data;
    }
}
Node buildTree(Scanner sc) {
    int val = sc.nextInt();
    if (val == -1) return null;

    Node root = new Node(val);
    root.left = buildTree(sc);
    root.right = buildTree(sc);
    return root;
}
```

**Visualization**

```
        1
      /   \
     2     3
```

## 2️⃣ Build Binary Tree from Array (Level Order)

**Index rules:**

Left child = 2*i + 1

Right child = 2*i + 2

**Code**

```
Node buildFromArray(int[] arr, int i) {
    if (i >= arr.length) return null;

    Node root = new Node(arr[i]);
    root.left = buildFromArray(arr, 2*i + 1);
    root.right = buildFromArray(arr, 2*i + 2);
    return root;
}
```

**Visualization (arr = {1,2,3,4,5})**

```
          1
        /   \
       2     3
     /  \
    4    5
```

## 3️⃣ Height, Depth, Level

**Height of Tree:** Height = longest path from node → leaf

```
int height(Node root) {
    if (root == null) return 0;
    return 1 + Math.max(height(root.left), height(root.right));
}
```

**Depth of a Node:** Depth = levels from root → node

```
int depth(Node root, int key, int d) {
    if (root == null) return -1;
    if (root.data == key) return d;

    int left = depth(root.left, key, d+1);
    if (left != -1) return left;

    return depth(root.right, key, d+1);
}
```

**Level of a Node:** Level = depth + 1

## 4️⃣ Count Nodes

### Total Nodes

```
int countNodes(Node root) {
    if (root == null) return 0;
    return 1 + countNodes(root.left) + countNodes(root.right);
}
```

### Leaf Nodes

```
int countLeaves(Node root) {
    if (root == null) return 0;
    if (root.left == null && root.right == null) return 1;
    return countLeaves(root.left) + countLeaves(root.right);
}
```

### Internal Nodes

```
int countInternal(Node root) {
    if (root == null) return 0;
    if (root.left == null && root.right == null) return 0;
    return 1 + countInternal(root.left) + countInternal(root.right);
}
```

## 5️⃣ Search in Binary Tree

(Simple BT search → not BST search)

```
boolean search(Node root, int key) {
    if (root == null) return false;
    if (root.data == key) return true;
    return search(root.left, key) || search(root.right, key);
}
```

## 6️⃣ Mirror (Invert) a Binary Tree

**Code**

```
Node mirror(Node root) {
    if (root == null) return null;

    Node temp = root.left;
    root.left = root.right;
    root.right = temp;

    mirror(root.left);
    mirror(root.right);

    return root;
}
```

**Example**

Before:

```
      1
     / \
    2   3
```

After:

```
      1
     / \
    3   2
```

## 7️⃣ Tree Diameter (Intro Version)

Diameter = longest path between any two nodes.

**Code**

```
int diameter(Node root) {
    if (root == null) return 0;
    int lh = height(root.left);
    int rh = height(root.right);
```

```
    int ld = diameter(root.left);
    int rd = diameter(root.right);

    return Math.max(lh + rh + 1, Math.max(ld, rd));
}
```

## 8️⃣ Insertion in General Binary Tree (Level Order Insert)

Used in CBT and almost CBT structures.

**Code**

```
Node insert(Node root, int key) {
    if (root == null) return new Node(key);

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node temp = q.poll();

        if (temp.left == null) {
            temp.left = new Node(key);
            return root;
        } else q.add(temp.left);

        if (temp.right == null) {
            temp.right = new Node(key);
            return root;
        } else q.add(temp.right);
    }
    return root;
}
```

**Result Example**

Insert 7 into:

```
      1
     / \
    2   3
   /
  4
```

New Tree:

```
      1
     / \
    2   3
   / \
  4   7
```

## 9️⃣ Deletion in General Binary Tree

**Steps:**

1. Find deepest node
2. Replace target with deepest node's value
3. Delete deepest node

**Code**

```
Node delete(Node root, int key) {
    if (root == null) return null;
    if (root.left == null && root.right == null) {
        if (root.data == key) return null;
        return root;
    }

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    Node keyNode = null, temp = null;

    while (!q.isEmpty()) {
        temp = q.poll();

        if (temp.data == key) keyNode = temp;

        if (temp.left != null) q.add(temp.left);
        if (temp.right != null) q.add(temp.right);
    }

    if (keyNode != null) {
        int x = temp.data;
        deleteDeepest(root, temp);
        keyNode.data = x;
    }
    return root;
}

void deleteDeepest(Node root, Node delNode) {
    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node temp = q.poll();
```

```
if (temp == delNode) {
            temp = null;
            return;
        }
        if (temp.left != null) {
            if (temp.left == delNode) {
                temp.left = null;
                return;
            }
            q.add(temp.left);
        }
        if (temp.right != null) {
            if (temp.right == delNode) {
                temp.right = null;
                return;
            }
            q.add(temp.right);
        }
    }
}
```

### Quick Summary

- Build trees (recursive / array)
- Measure height / depth / levels
- Count all types of nodes
- Search nodes
- Mirror a tree
- Compute basic diameter
- Insert and delete using level-order logic

## 5.5 — Complete Binary Tree (CBT)

The backbone of heaps. The most structured binary tree.

### 1️⃣ What is a Complete Binary Tree?

A Complete Binary Tree (CBT) is a binary tree where:

- Every level, except possibly the last, is completely filled
- The last level has nodes as **left as possible**

This makes it the most “compact” form of binary tree.

**Example of CBT:**

```
        10
      /    \
    20      30
   /  \    /
 40   50  60
```

**Not a CBT:**

(Last level not left-filled)

```
        10
      /    \
    20      30
     \
      50
```

## 2 Why CBT? (Motivation & Importance)

CBT has maximum space efficiency and supports:

- Fast indexing (array representation)
- Perfect for **Heaps** (Min-Heap / Max-Heap)
- Ideal for priority queues
- Great for BFS-based operations

A CBT ensures **minimal height**, guaranteeing **O(log n)** insert/delete in a heap.

## 3 Properties of a CBT

- Height = $\lfloor \log_2 n \rfloor$
- Levels filled left to right
- Minimal height for given nodes
- Perfect structure for array storage
- Index-based navigation becomes natural

## 4 Array Indexing Formulas (Core CBT Property)

**For a node at index i (1-based indexing):**

- Left child = `2 * i`
- Right child = `2 * i + 1`
- Parent = `i / 2`

**For 0-based indexing:**

- Left = `2*i + 1`
- Right = `2*i + 2`
- Parent = `(i - 1) / 2`

These formulas are why **heaps use arrays**.

## 5️⃣ Array Representation of CBT (With Visualization)

**Example CBT:**

```
        10
      /    \
    20      30
   /  \    /
 40   50  60
```

**Array Form (1-based indexing):**

```
Index:   1   2   3   4   5   6
Value:  10  20  30  40  50  60
```

**Array Form (0-based indexing):**

```
Index:   0   1   2   3   4   5
Value:  10  20  30  40  50  60
```

Perfect compact usage — **no gaps, no nulls**.

## 6️⃣ Linked List Representation of CBT

**Node structure:**

```
class Node {
    int data;
    Node left, right;
    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
```

BUT — CBT operations in linked form must follow **level-order insertion rules**.

## 7️⃣ CRUD Operations on CBT (Core Section)

(all operations follow BFS order)

### ✔ A. Insert into CBT

Always insert at the first empty position in level order
→ ensures tree stays complete.

**Algorithm (Linked list version):**

1. Use a **queue** for level-order traversal
2. First node whose `left` is null → insert
3. Else if `right` is null → insert
4. Else continue BFS

**Code:**

```
Node insert(Node root, int value) {
    Node newNode = new Node(value);

    if (root == null) return newNode;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node temp = q.poll();

        if (temp.left == null) {
            temp.left = newNode;
            return root;
        }
        else if (temp.right == null) {
            temp.right = newNode;
            return root;
        }

        q.add(temp.left);
        q.add(temp.right);
    }

    return root;
}
```

### ✔ B. Delete from CBT (Delete Deepest Node)

CBT deletion is **not BST deletion**.
Here's how CBT deletion works:

1. Find the last node in level order
2. Replace target node's value with last node's value
3. Delete the deepest node

**Code:**

```
Node delete(Node root, int key) {
    if (root == null) return null;

    if (root.left==null && root.right==null) {
        return (root.data == key) ? null : root;
    }

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    Node keyNode = null, temp = null;

    while (!q.isEmpty()) {
        temp = q.poll();

        if (temp.data == key)
            keyNode = temp;

        if (temp.left != null) q.add(temp.left);
        if (temp.right != null) q.add(temp.right);
    }

    if (keyNode != null) {
        int x = temp.data;    // deepest node value
        deleteDeepest(root, temp);
        keyNode.data = x;
    }
    return root;
}

void deleteDeepest(Node root, Node delNode) {
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    while (!q.isEmpty()) {
        Node temp = q.poll();

        if (temp == delNode) {
            temp = null;
            return;
        }
        if (temp.left != null) {
            if (temp.left == delNode) {
                temp.left = null;
                return;
            } else q.add(temp.left);
        }
```

```
 if (temp.right != null) {
            if (temp.right == delNode) {
                temp.right = null;
                return;
            } else q.add(temp.right);
        }
    }
}
```

✔ **C. Build CBT from Array**

```
Node build(int[] arr, int i) {
    if (i >= arr.length) return null;

    Node root = new Node(arr[i]);
    root.left = build(arr, 2*i + 1);
    root.right = build(arr, 2*i + 2);

    return root;
}
```

✔ D. Build CBT from Level-Order Input

Use a queue — read one element → attach as left/right.

## 8️⃣ Traversal of CBT

CBT uses the same DFS and BFS traversal as normal BT.

- Preorder: Root L R
- **Inorder**: L Root R
- Postorder: L R Root
- **Level Order**: BFS (core traversal for CBT)

Since the tree is complete, level-order is beautifully compact.

## 9️⃣ CBT → Heap Connection (THE MAIN REASON CBT MATTERS)

A **binary heap** (min-heap or max-heap) requires:

- A Complete Binary Tree
- AND the heap property (ordering)

**CBT guarantees:**

- Minimal height → O(log n) insert/delete
- Perfect array mapping → faster memory access
- No fragmentation → cache efficiency

Without CBT, Heaps cannot exist.

## 🔟 Comparison: CBT vs BT vs BST

| Feature | BT | BST | CBT |
|---|---|---|---|
| Structure | Arbitrary | Ordered | Level-filled |
| Insert position | Anywhere | Based on value | Next free spot |
| Delete rules | Many | Based on BST property | Replace with deepest |
| Guaranteed height | No | No | Yes → O(log n) |
| Array use | Not ideal | No | Perfect |
| Used in | General tree | Searching | Heaps, PQ |

### Quick Summary:

- CBT is a **binary tree filled left-to-right**, all levels except last are full
- Core of **Heaps** and all priority queues
- Array representation is extremely natural
- Insert = BFS-first-null
- Delete = replace with deepest node
- Traversals are same as BT
- CBT → minimal height → optimal operations

**Mantra:**

"CBT gives structure. Heap gives power. Together they build priority."

# 5.6 — Binary Search Tree (BST)

The tree that brings order to hierarchy.

## 1️⃣ What is a Binary Search Tree?

A Binary Search Tree (BST) is a Binary Tree with an additional ordering property:

For every node with value **X**:

- All nodes in its **left subtree** have values **< X**
- All nodes in its **right subtree** have values **> X**
- No duplicates (in standard BST)

**Visual:**

```
        50
       /  \
     30    70
    / \    / \
  20  40  60  80
```

- This simple ordering property makes BST incredibly powerful.

## 2️⃣ Why BST?

**BST allows:**

- **O(log n)** average time for search
- **O(log n)** average time for insert
- **O(log n)** average time for delete

Because each step eliminates half the tree, just like Binary Search.

But… only if BST is **balanced**.

## 3️⃣ Node Structure (Linked Representation)

```
class Node {
    int data;
    Node left, right;

    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
```

A BST is always built using Linked Nodes.

## 4️⃣ Insertion in BST (Iterative Version)

Insert `value` into its correct position.

**Rules:**

- If value < node → go left
- If value > node → go right
- Insert where null is found

**Java Code:**

```
Node insert(Node root, int key) {
    if (root == null)
        return new Node(key);
    Node curr = root;
    while (true) {
        if (key < curr.data) {
            if (curr.left == null) {
                curr.left = new Node(key);
                break;
            }
```

```
curr = curr.left;
        }
        else {
            if (curr.right == null) {
                curr.right = new Node(key);
                break;
            }
            curr = curr.right;
        }
    }
    return root;
}
```

## 5 Search in BST

Exactly like binary search on arrays.

**Java Code:**

```
boolean search(Node root, int key) {
    while (root != null) {
        if (key == root.data) return true;
        else if (key < root.data) root = root.left;
        else root = root.right;
    }
    return false;
}
```

## 6 Deletion in BST (All 3 Cases)

This is the MOST important part of BST.

We handle three cases:

**Case 1 — Deleting a Leaf Node**

Example: Delete 20

```
    30
   /
 20
```

→ Just remove.

**Case 2 — Deleting a Node with One Child**

Example:

```
    30
   /
 20
   \
    25
```

Replace node with its child.

**Case 3 — Deleting a Node with Two Children**

MOST IMPORTANT CASE

**Example: Delete 50**

```
        50
       /  \
     30    70
          /
         60
```

**Steps:**

1. Find Inorder Successor (smallest in right subtree)
2. Replace node → successor value
3. Delete successor node

**Java Code for Deletion**

```
Node delete(Node root, int key) {
    if (root == null) return null;
    if (key < root.data)
        root.left = delete(root.left, key);
    else if (key > root.data)
        root.right = delete(root.right, key);
    else {
        // Case 1: No child
        if (root.left == null && root.right == null)
            return null;

        // Case 2: One child
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        // Case 3: Two children
        Node successor = findMin(root.right);
        root.data = successor.data;
        root.right = delete(root.right, successor.data);
    }
    return root;
}

Node findMin(Node root) {
    while (root.left != null)
        root = root.left;
    return root;
}
```

## 7️⃣ Traversal Property

Inorder traversal of a BST → Always Sorted

**Example BST:**

```
        50
       /  \
     30    70
    / \    / \
  20  40  60  80
```

**Inorder Output:** 20 30 40 50 60 70 80

**This SINGLE PROPERTY powers:**

- Searching
- Sorting
- Range queries
- Successor/Predecessor
- Validating BST

## 8️⃣ Finding Successor & Predecessor

Successor = next larger value

Predecessor = next smaller value

Inorder Successor (right subtree)

successor = min(node.right)

Inorder Predecessor (left subtree)

predecessor = max(node.left)

**Code:**

```
Node getSuccessor(Node root) {
    root = root.right;
    while (root.left != null)
        root = root.left;
    return root;
}
Node getPredecessor(Node root) {
    root = root.left;
    while (root.right != null)
        root = root.right;
    return root;
}
```

## 9️⃣ Validate BST

**Definition:**
**Every node must satisfy:**

min < node.data < max

**Code Using Range Check:**

```
boolean isValidBST(Node root, long min, long max) {
    if (root == null) return true;

    if (root.data <= min || root.data >= max)
        return false;

    return isValidBST(root.left, min, root.data) &&
           isValidBST(root.right, root.data, max);
}
```

Call with: isValidBST(root, Long.MIN_VALUE, Long.MAX_VALUE);

## 🔟 Balanced vs Unbalanced BST (Intro Only)

**Unbalanced BST**

Degenerates into a linked list (worst case):

```
10
  \
   20
     \
      30
        \
         40
```

Worst case complexity becomes:

Search: O(n)

Insert: O(n)

Delete: O(n)

**Balanced BST**

Keeps height ≈ log n:

Includes:

- AVL Trees
- Red-Black Trees
- Splay trees (advanced)

We only introduce the concept here — full chapters later.

### Quick Summary

- BST enforces **sorted structure** using L < Root < R
- Average complexity: O(log n)
- Worst-case: O(n) (skewed)
- Insertion → left or right
- Search → left or right
- Deletion → 3 cases
- Inorder traversal → always sorted
- Successor = leftmost in right subtree
- Predecessor = rightmost in left subtree
- BST must be validated via **range check**
- Balanced BSTs solve skewing

### Mantra:

"BST is the perfect marriage of hierarchy and order."

## 5.7 — Applications of Binary Search Trees (BST)

Where ordered structure meets real-world efficiency.

### 1️⃣ Fast Searching (Primary Purpose of BST)

A BST stores data in **sorted order**, so searching becomes extremely efficient.

**Property Used:**

Left subtree < Root < Right subtree

**Why It Matters:**

With this ordering property, searching becomes exactly like **binary search on trees**.

**Time:**

- Balanced BST → O(log n)
- Unbalanced BST → O(n)

**Real Use:**

- Searching employee IDs
- Searching roll numbers
- Searching dictionary words
- Searching contact names

BST = Fast search with ordered structure.

## 2️⃣ Dictionary / Symbol Table Implementation

**A Symbol Table maps:**

identifier → data/value

**Example:**

int x = 10;

float pi = 3.14;

**A BST helps store:**

- variable names
- function names
- constants
  in sorted order for **quick lookup**.

**Where used:**

- Compilers
- Interpreters
- Programming languages symbol tables

**Why BST?**

- Sorted
- Fast insertion
- Fast search
- Good for dynamically changing data

## 3️⃣ Set / Map Data Structure (Conceptual Foundation)

Although languages use **balanced BSTs (like Red-Black Trees)** for modern Sets/Maps,
the conceptual foundation comes from the **BST ordering property**.

**Example:**

- Java TreeSet, TreeMap → use Red-Black Tree
- C++ set, map → use Balanced BST
- Python btree (external module)

**What BST gives:**

- Unique keys
- Sorted order
- Logarithmic operations

## 4️⃣ Auto-Suggestion / Auto-Completion

**When the user types:** app

**We need to show:**

apple

appstore

application

BST helps store words in **sorted order**, so we can:

- Search prefix in O(log n)
- Walk through in sorted order
- Collect suggestions efficiently

**Used In:**

- Google search predictions
- IDE autocompletion
- Contact name suggestion in mobiles
- Browser autocomplete

## 5️⃣ Range Queries

**Example:**

"Print all values between 20 and 60."

**In BST:**

- Inorder traversal gives sorted order
- We can skip entire subtrees that don't fall in the range

**Time:**

O(k + log n)

where k = number of keys in range

**Applications:**

- Databases (range filters)
- Data analytics
- Searching values within time intervals
- Revenue between [lower, upper] bounds

## 6️⃣ Storing Sorted Data Efficiently

If we perform an **inorder traversal** of a BST:

Left → Root → Right

We get sorted order of all stored keys.

**How it's used:**

- Sorting using BST (Tree Sort)
- Real-time sorted list maintenance
- Maintaining leaderboard
- Maintaining sorted event logs

**Complexity:**

BST Insertion: O(log n)

BST Inorder: O(n)

**Contrast with:**

- Array insertion: O(n)
- Linked list sorting: O(n log n)

BST keeps your data sorted **as you insert it**.

## 7️⃣ File System & Directory Structures (Conceptual)

Many file systems internally use **tree-like ordered structures**.
Though they often use B-Trees/B+Trees, the *core idea* comes from BST-style ordering:

- Left subtree < root < right subtree
- Efficient insertion/deletion
- Efficient search

## 8️⃣ Memory Allocation (Buddy System – Conceptual)

OS memory allocation sometimes uses tree-based splitting logic:

- Request memory
- Split blocks
- Merge buddies
- Track free blocks

BST-like ordering helps manage variable-sized memory blocks.

(Not a pure BST, but uses ordered binary tree logic.)

## 9️⃣ Gaming & Decision-Based Applications

Game AI uses tree structures to store:

- Choices
- Scores
- Game states

BST-style ordering helps prune useless branches and speed up decisions.

## Quick Summary

| Application | Why BST? |
|---|---|
| Fast search | Ordered structure → O(log n) |
| Symbol table | Quick lookup of identifiers |
| Set/Map | Sorted keys, uniqueness, log-time ops |
| Auto-suggestion | Prefix search + sorted traversal |
| Range queries | Skip irrelevant subtrees |
| Sorted storage | Inorder → sorted data |
| Game trees | Efficient pruning & decisions |
| Memory mgmt | Ordered block organization |

# 5.8 — Binary Tree vs BST vs CBT

The ultimate comparison of the three foundational tree structures.

## 1️⃣ Why This Comparison Matters

Binary Tree, BST, and CBT are closely related —but each has a **very different purpose**, **different rules**, and **different performance guarantees**.

**If students understand this comparison, they will always choose the right tree for the right problem.**
**This is essential for:**

- DSA intuition
- System design
- Interview problem-solving
- Understanding Heaps, AVL, Red-Black Trees later

## 2️⃣ What Are They? (Short Definitions)

**Binary Tree (BT)**

A generic tree where each node has **at most 2 children**. No ordering rule. No shape rule.

**Binary Search Tree (BST)**

An ordered binary tree where:

Left subtree < Root < Right subtree

Inorder traversal → **sorted order**.

**Complete Binary Tree (CBT)**

A tree where:

- All levels are completely filled **except possibly the last**
- Last level is filled **left to right**

Foundation of **Heaps**.

## 3 Structural Rules

| Tree Type | Max Children | Shape Constraint | Ordering Constraint |
|---|---|---|---|
| BT | 2 | ❌ None | ❌ None |
| BST | 2 | ❌ None | ✔ Left < Root < Right |
| CBT | 2 | ✔ Must be complete | ❌ None |

## 4 Visual Intuition

### Binary Tree (BT)

Any shape:

```
      A
     / \
    B   C
       /
      D
```

### Binary Search Tree (BST)

Ordered shape:

```
        40
      /    \
   25       60
  /  \     /  \
10   30   50   70
```

### Complete Binary Tree (CBT)

Perfectly packed left to right:

```
         10
        /  \
      20    30
     / \   /
   40  50 60
```

## 5 Insertion Rules

### BT

Anywhere (general insertion).

### BST

Left < Root < Right (ordered insert).

### CBT

Always inserted in the next available position in level order.

## 6️⃣ Deletion Rules

**BT**

Delete leaf or promote children (general rules).

**BST**

3 cases:

- No child
- One child
- Two children (replace with inorder successor/predecessor)

**CBT**

Replace node with **last level node**, then delete last node.

## 7️⃣ Traversal Outcomes

| Traversal | BT | BST | CBT |
|---|---|---|---|
| Inorder | No guarantee | Sorted | Arbitrary |
| Preorder | Arbitrary | Not sorted | Arbitrary |
| Postorder | Arbitrary | Not sorted | Arbitrary |
| Level Order | Arbitrary | Ordered by tree structure | Shows CBT shape clearly |

Only BST gives **sorted inorder**.

## 8️⃣ Performance Comparison

| Operation | BT | BST | CBT |
|---|---|---|---|
| Search | O(n) | O(h), best O(log n) | O(n) |
| Insert | O(n) | O(h), best O(log n) | ✔ O(1) amortized |
| Delete | O(n) | O(h), best O(log n) | ✔ O(1) amortized |
| Traversal | O(n) | O(n) | O(n) |

Note:
BST worst case becomes **O(n)** when unbalanced.

## 9️⃣ Use Cases

**Binary Tree**

- Expression trees
- Parse trees
- Generic hierarchical data
- File directory structure

**Binary Search Tree**

- Searching
- Dictionary/Map data
- Auto-suggestion systems
- Range queries

**Complete Binary Tree**

- **Heaps** (Max/Min Heap)
- Priority queues
- Scheduling systems
- Memory allocation structures

## 10 Memory Representation

| Tree | Best Representation |
|---|---|
| BT | Linked list nodes |
| BST | Linked list nodes |
| CBT | Array (ideal for Heap) |

CBT → array is extremely cache-efficient.

## 11 Summary Table (Super-Condensed)

| Feature | BT | BST | CBT |
|---|---|---|---|
| Children | ≤ 2 | ≤ 2 | ≤ 2 |
| Shape | Free | Free | Structured |
| Ordering | None | Left < Root < Right | None |
| Search | Slow | Fast | Slow |
| Insert | Free | Ordered | Strict level order |
| Memory | Linked | Linked | Array best |
| Ideal Use | Hierarchy | Search | Heaps |

## 12 Final Essence

Binary Tree gives structure, BST gives order, CBT gives shape.

BT = flexible
BST = searchable
CBT = heap-friendly & space-efficient

## 5.9 — Real-World Applications of Trees

Where theory grows into real systems.

Trees are not just a data structure — they are the foundation of how modern software organizes, searches, compiles, routes, predicts, and optimizes data.

Trees appear everywhere because:

- They model **hierarchies**
- They allow logarithmic search
- They support structured decisions
- They provide balanced organization
- They reduce linear scans dramatically

Let's explore the most important real-world places where trees power the world.

### 1️⃣ File Systems (Folders & Subfolders)

Your operating system's file explorer:

```
Root
 ├── Documents
 │    ├── Projects
 │    └── Notes
 ├── Downloads
 └── Pictures
```

✔ Natural tree structure
✔ Directory = internal node
✔ File = leaf node

**Why Tree?**

- Perfect for hierarchy
- Fast navigation (DFS/BFS)
- Easy expansion in any branch

### 2️⃣ HTML / XML DOM Trees

Every webpage is a **tree**:

```
<html>
  <body>
    <div>
      <p> Hello </p>
    </div>
  </body>
</html>
```

✔ Parent–child relationships
✔ DOM traversal = Tree traversal
✔ Used by browsers to render pages

**Why Tree?**

- Nested structure
- Allows depth-based styling (CSS)
- Easy for parsing algorithms

## 3️⃣ Compilers & Syntax Trees

Compilers convert programs into parse trees / abstract syntax trees (AST).

**Example:**

Expression: a + b * c

**AST:**

```
      +
    /   \
   a     *
        / \
       b   c
```

✔ Used in syntax analysis
✔ Used in optimization
✔ Used in code generation

**Why Tree?**

- Represents grammar structure
- Allows recursive evaluation
- Perfect for expression semantics

## 4️⃣ Routing Algorithms & Networking

**Routers use tree structures to determine:**

- Shortest path
- Forwarding tables
- Route hierarchies

**For example:**

- **Trie** structures are used in IP routing
- MST (Minimum Spanning Trees) optimize networks
- **Routing trees** make packet delivery efficient

## 5️⃣ Expression Trees

Used in calculators, compilers, and interpreters.

**Example:**

(5 + 2) * (7 - 3)

```
        *
      /   \
    +       -
   / \     / \
  5   2   7   3
```

✔ **Tree traversal = evaluation**

- Postorder → Evaluation
- Inorder → Original expression
- Preorder → Prefix notation

**Why Tree?**

- Represents operators with correct precedence
- Easy recursive evaluation

## 6️⃣ Decision Trees (Machine Learning)

Decision Trees are one of the most popular ML models.

**Example:**

```
          Is Age > 25?
         /             \
      Yes               No
     /                   \
 Income>50k?             Student?
```

✔ Used for prediction
✔ Used in classification
✔ Models real human decision-making

**Why Tree?**

- Each decision → branch
- Final answer → leaf
- Easy to visualize

## 7️⃣ Game Trees (AI & Strategy Games)

Games like Chess, Go, Tic-Tac-Toe rely on **minimax trees**.

**Example:**

Root = Current Board

Children = Next possible moves

Grandchildren = Opponent responses

✔ Used by AI to explore possibilities
✔ BFS/DFS used to search move space
✔ Alpha-Beta pruning trims branches

**Why Tree?**

- Perfect for state exploration
- Game decisions are branching

## 8️⃣ Trie Structures (Prefix Trees)

**Used in:**

- Autocomplete
- Spell check
- Dictionary lookup
- Search suggestions (Google)

**Example of a trie storing:**

cat, cab, car

```
    c
    |
    a
  /  |  \
 t   b   r
```

✔ O(L) search time
✔ Perfect for prefix queries

## 9️⃣ Database Indexing (B-Trees & B+ Trees)

Every modern database (MySQL, MongoDB, PostgreSQL) uses:

- B-Tree
- B+ Tree

**Why Trees?**

- Very few disk accesses
- Balanced structure → O(log n) search
- Perfect for indexing millions of records

✔ Searching
✔ Inserting
✔ Deleting
✔ Maintaining sorted data

Databases would be unbearably slow without tree-based indexing.

## 10️⃣ Hierarchical Data Modeling Everywhere

**Trees appear in:**

- Organization charts
- UI menus
- Category/sub-category systems
- XML/JSON structure
- Knowledge graphs (tree-subtrees)
- Messaging queues (broker trees)

Whenever we need hierarchy → we need trees.

## Quick Summary

| Area | Tree Structure Used | Why? |
|---|---|---|
| File System | General Tree | Hierarchy |
| Web Browsers | DOM Tree | Nested HTML |
| Compilers | AST / Parse Trees | Expression structure |
| Networking | Routing Trees, Tries | Fast lookup |
| Databases | B-Trees, B+ Trees | Indexing |
| AI | Game Trees | Decision exploration |
| ML | Decision Trees | Predictive modeling |
| Search Engines | Trie | Auto-suggest |
| OS | Process trees | Parent–child processes |

### Mantra

"Trees are everywhere — wherever data branches, grows, or makes decisions, a tree silently powers the system."